



Disaster Event Classification using K-Nearest Neighbors

Problem Statement

Natural disasters vary in severity and impact, making it important to identify whether a disaster event can be classified as major or not. This project aims to classify major disaster events using the K-Nearest Neighbors (K-NN) algorithm by analyzing disaster characteristics such as severity level, affected population, economic loss, response time, aid provided, and infrastructure damage. The model supports early assessment and improved disaster management decision-making.

Steps to be followed

1. Importing Required Libraries
2. Load Dataset
3. Data Preprocessing
4. Checking Null values
5. Data Wrangling (Filling Missing values)
6. Data Visualization
7. Label Encoding
8. Feature Selection (x,y)
9. Train-Test Split
10. Feature Scaling
11. K-NN Model Training and Prediction
12. Model Evaluation (Accuracy Score, Classification Report)
13. Confusion Matrix
14. Cross Validation Score
15. GridSearchCV and RandomizedSearchCV
16. ROC-AUC Score and ROC-AUC Curve
17. Improving Model Performance with Advanced Algorithms

Importing Libraries

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
```

```
import math
```

```
In [53]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedS
from sklearn.metrics import accuracy_score, precision_score, classification_rep
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, StandardScaler,
import warnings
warnings.filterwarnings('ignore')
```

Loading the Dataset

```
In [3]: df=pd.read_csv(r"C:\Users\HP\Downloads\synthetic disaster events 2025 updated.
df
```

```
Out[3]:
```

	disaster_type	location	latitude	longitude	Year	severity_level	af
0	Wildfire	Chile	-34.681672	-71.819529	2025	8.0	
1	Hurricane	India	22.128569	78.023951	2023	5.0	
2	Volcanic Eruption	Italy	42.316058	11.031447	2023	NaN	
3	Drought	Chile	-33.436253	-69.984615	2024	NaN	
4	Volcanic Eruption	Turkey	39.400977	37.006822	2023	8.0	
...	...	...	...	...	...	...	...
19995	NaN	India	19.962542	82.590546	2025	6.0	
19996	Volcanic Eruption	Japan	35.853659	138.522874	2025	NaN	
19997	Hurricane	Philippines	13.720136	121.306074	2025	8.0	
19998	Volcanic Eruption	India	19.959071	77.272460	2023	7.0	
19999	NaN	Chile	-36.842195	-68.613505	2023	6.0	

20000 rows × 12 columns

Data Preprocessing

```
In [4]: df.shape
```

```
Out[4]: (20000, 12)
```

```
In [5]: df.head()
```

```
Out[5]:
```

	disaster_type	location	latitude	longitude	Year	severity_level	affected_
0	Wildfire	Chile	-34.681672	-71.819529	2025	8.0	
1	Hurricane	India	22.128569	78.023951	2023	5.0	
2	Volcanic Eruption	Italy	42.316058	11.031447	2023	NaN	
3	Drought	Chile	-33.436253	-69.984615	2024	NaN	
4	Volcanic Eruption	Turkey	39.400977	37.006822	2023	8.0	

```
In [6]: df.tail()
```

```
Out[6]:
```

	disaster_type	location	latitude	longitude	Year	severity_level	af
19995	NaN	India	19.962542	82.590546	2025	6.0	
19996	Volcanic Eruption	Japan	35.853659	138.522874	2025	NaN	
19997	Hurricane	Philippines	13.720136	121.306074	2025	8.0	
19998	Volcanic Eruption	India	19.959071	77.272460	2023	7.0	
19999	NaN	Chile	-36.842195	-68.613505	2023	6.0	

```
In [7]: df.sample()
```

```
Out[7]:
```

	disaster_type	location	latitude	longitude	Year	severity_level	affect
13258	NaN	Italy	41.445123	14.508406	2022	NaN	

Checking Null Values

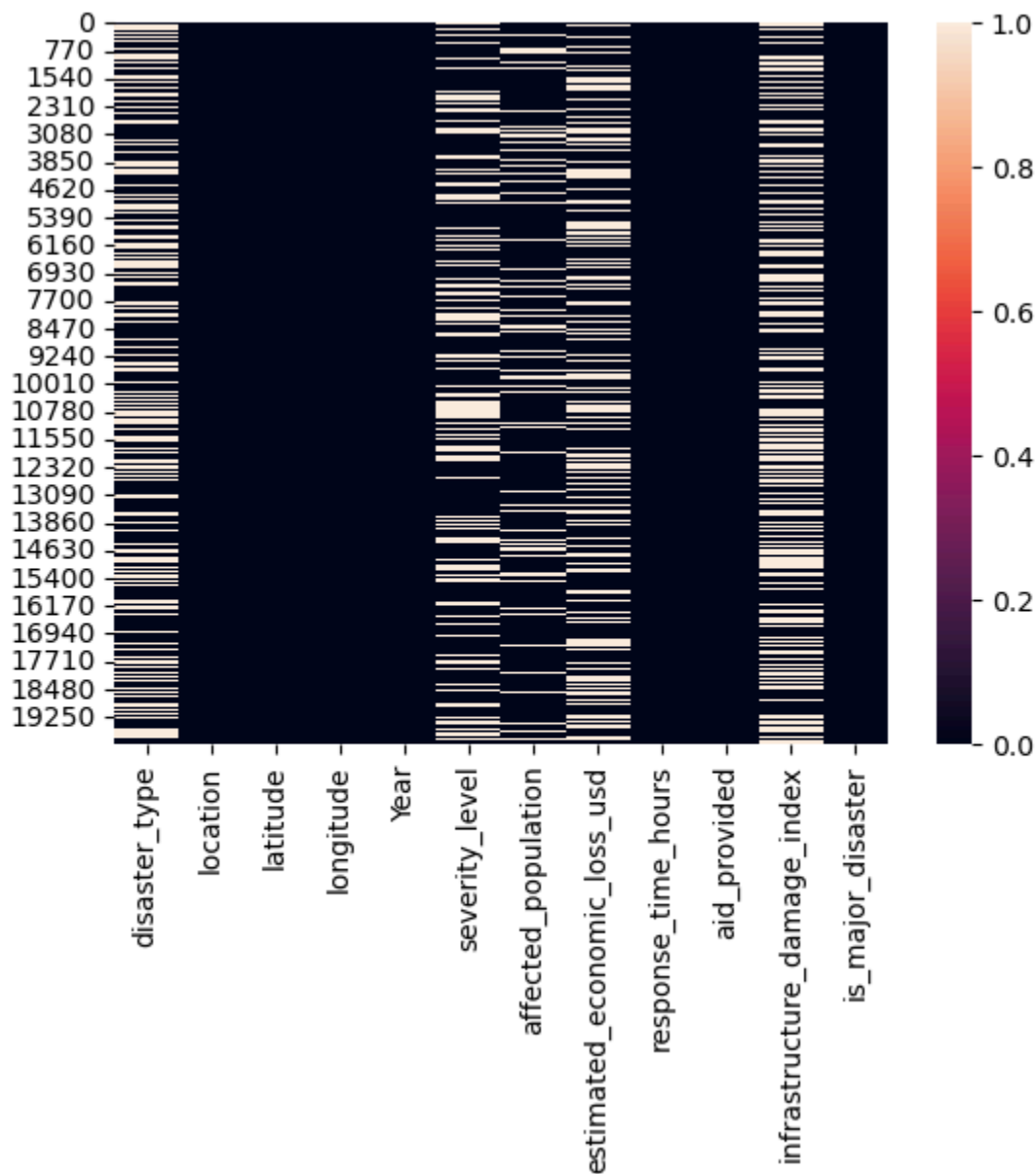
Missing values are identified to ensure data quality before training the model.

```
In [8]: df.isnull().sum()
```

```
Out[8]: disaster_type      7884
location      0
latitude      0
longitude     0
Year          0
severity_level 5111
affected_population 3410
estimated_economic_loss_usd 6596
response_time_hours 0
aid_provided  0
infrastructure_damage_index 8606
is_major_disaster 0
dtype: int64
```

```
In [9]: sns.heatmap(df.isnull())
```

```
Out[9]: <Axes: >
```



```
In [10]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   disaster_type                         12116 non-null  object
1   location                             20000 non-null  object
2   latitude                             20000 non-null  float64
3   longitude                             20000 non-null  float64
4   Year                                 20000 non-null  int64
5   severity_level                       14889 non-null  float64
6   affected_population                  16590 non-null  float64
7   estimated_economic_loss_usd         13404 non-null  float64
8   response_time_hours                  20000 non-null  float64
9   aid_provided                         20000 non-null  object
10  infrastructure_damage_index          11394 non-null  float64
11  is_major_disaster                    20000 non-null  int64
dtypes: float64(7), int64(2), object(3)
memory usage: 1.8+ MB
```

```
In [11]: df.describe()
```

```
Out[11]:
```

	latitude	longitude	Year	severity_level	affected_popula
count	20000.000000	20000.000000	20000.000000	14889.000000	16590.00
mean	18.749811	41.976629	2023.941200	5.480220	27671.37
std	25.026759	82.535922	0.850399	2.875231	16036.91
min	-43.127383	-102.133704	2022.000000	1.000000	0.00
25%	3.560059	8.346726	2023.000000	3.000000	14710.50
50%	24.361006	74.600322	2024.000000	5.000000	27603.50
75%	38.285895	117.460836	2025.000000	8.000000	40059.00
max	48.452161	145.902669	2025.000000	10.000000	75147.00

```
In [12]: df.dtypes
```

```
Out[12]: disaster_type      object
         location           object
         latitude          float64
         longitude         float64
         Year              int64
         severity_level     float64
         affected_population float64
         estimated_economic_loss_usd float64
         response_time_hours float64
         aid_provided       object
         infrastructure_damage_index float64
         is_major_disaster   int64
         dtype: object
```

```
In [13]: df.columns
```

```
Out[13]: Index(['disaster_type', 'location', 'latitude', 'longitude', 'Year',
               'severity_level', 'affected_population', 'estimated_economic_loss_usd',
               'response_time_hours', 'aid_provided', 'infrastructure_damage_index',
               'is_major_disaster'],
              dtype='object')
```

```
In [ ]:
```

```
In [ ]:
```

Data Wrangling (Filling Null/Missing Values)

Some numerical and categorical features contain missing values which will be handled using median and mode imputation respectively.

```
In [14]: num_cols=['severity_level', 'affected_population', 'estimated_economic_loss_usd']
         df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

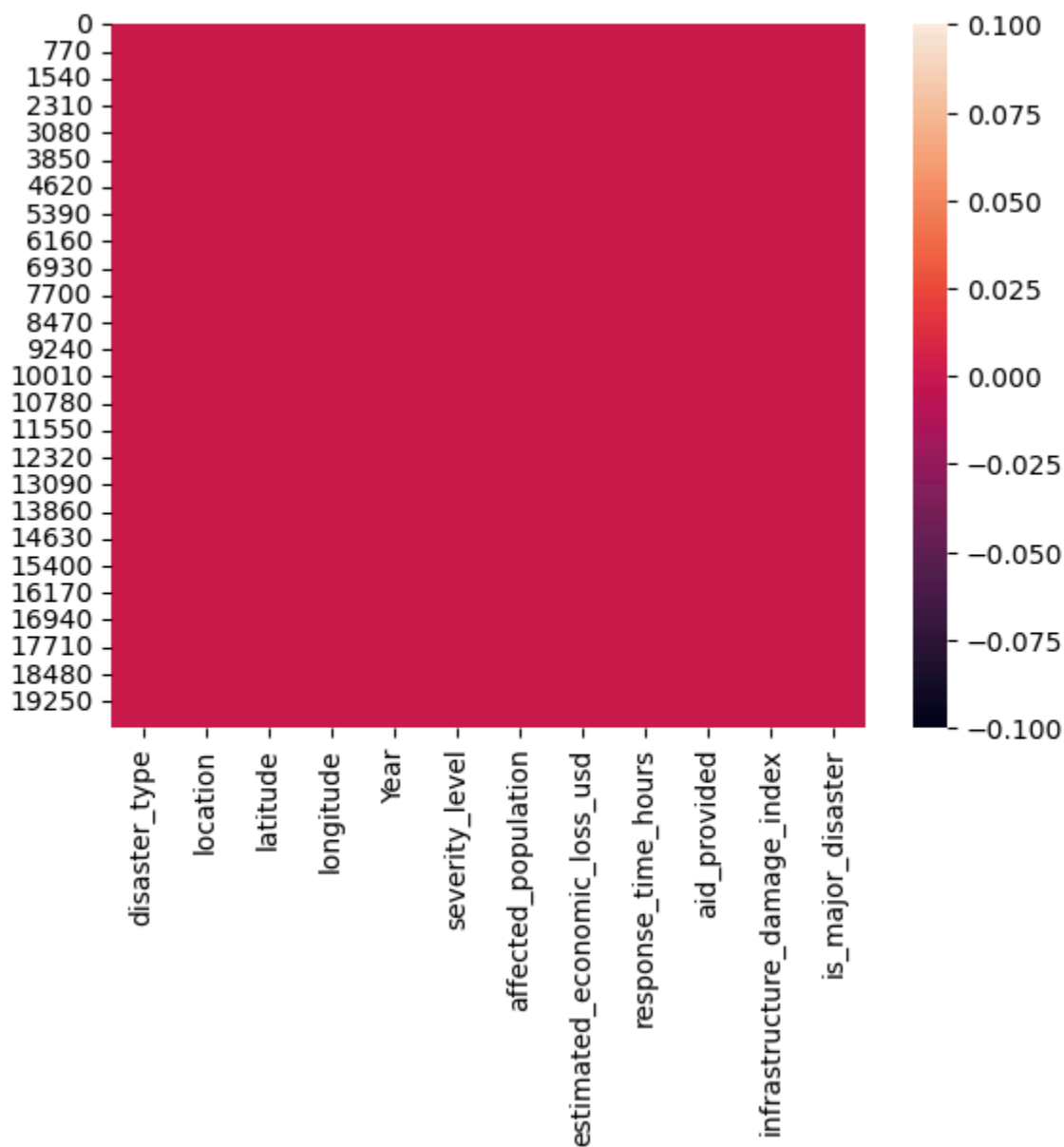
```
In [15]: cat_cols=['disaster_type']
         df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
```

```
In [16]: df.isnull().sum()
```

```
Out[16]: disaster_type      0
location      0
latitude      0
longitude     0
Year          0
severity_level 0
affected_population 0
estimated_economic_loss_usd 0
response_time_hours 0
aid_provided  0
infrastructure_damage_index 0
is_major_disaster 0
dtype: int64
```

```
In [17]: sns.heatmap(df.isnull())
```

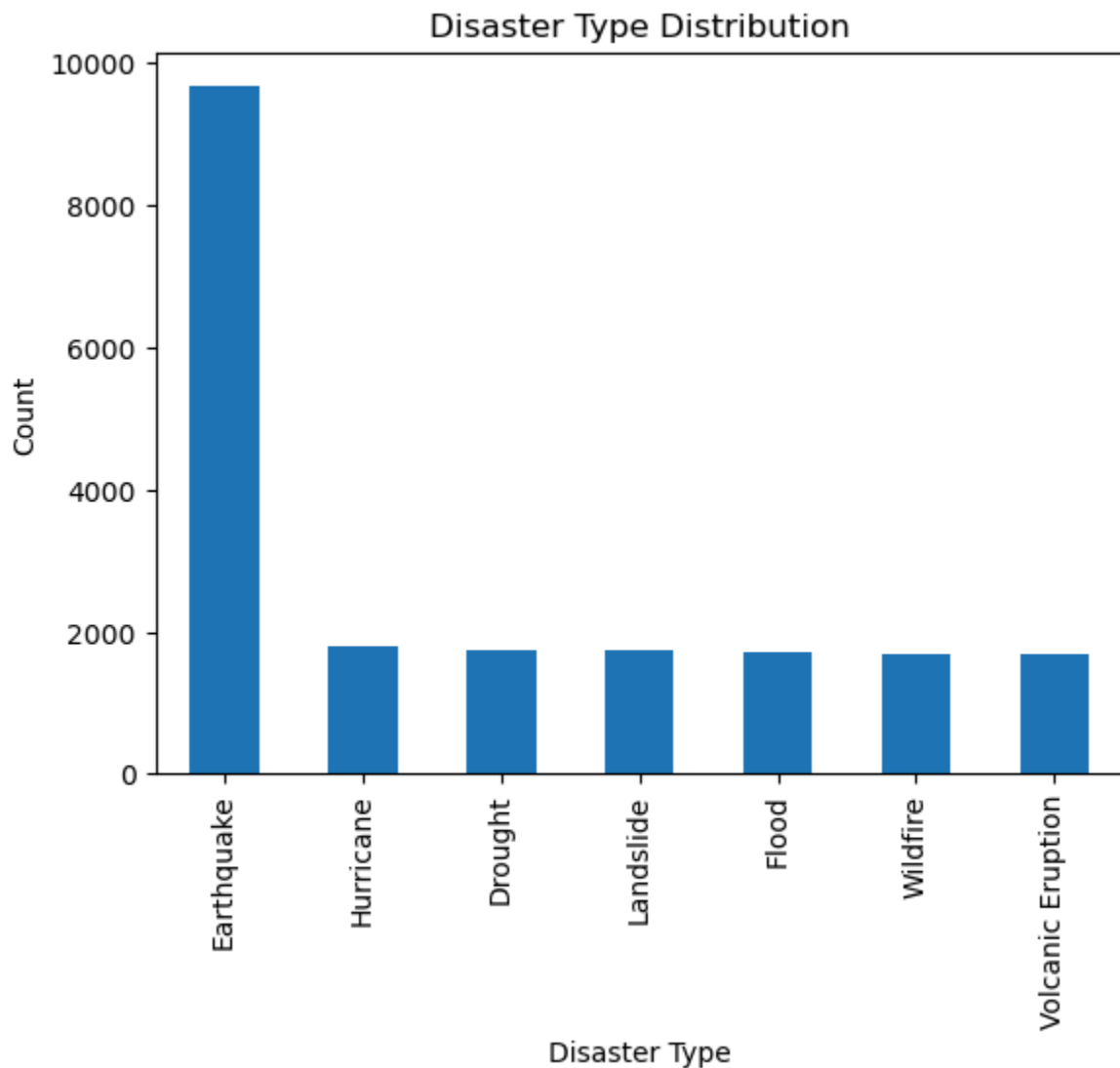
```
Out[17]: <Axes: >
```



In []:

Data Visualization

```
In [18]: plt.figure()
df['disaster_type'].value_counts().plot(kind='bar')
plt.title("Disaster Type Distribution")
plt.xlabel("Disaster Type")
plt.ylabel("Count")
plt.show()
```



Earthquakes represent the most common disaster type in the dataset, indicating higher occurrence or reporting frequency.

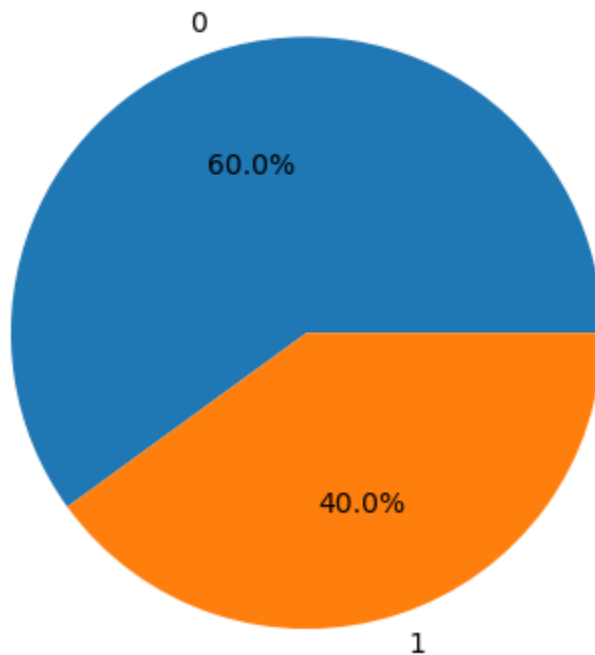
```
In [19]: plt.figure()
df['is_major_disaster'].value_counts().plot(
    kind='pie', autopct='%1.1f%%')
```

```

)
plt.title("Major vs Non-Major Disasters")
plt.ylabel("")
plt.show()

```

Major vs Non-Major Disasters

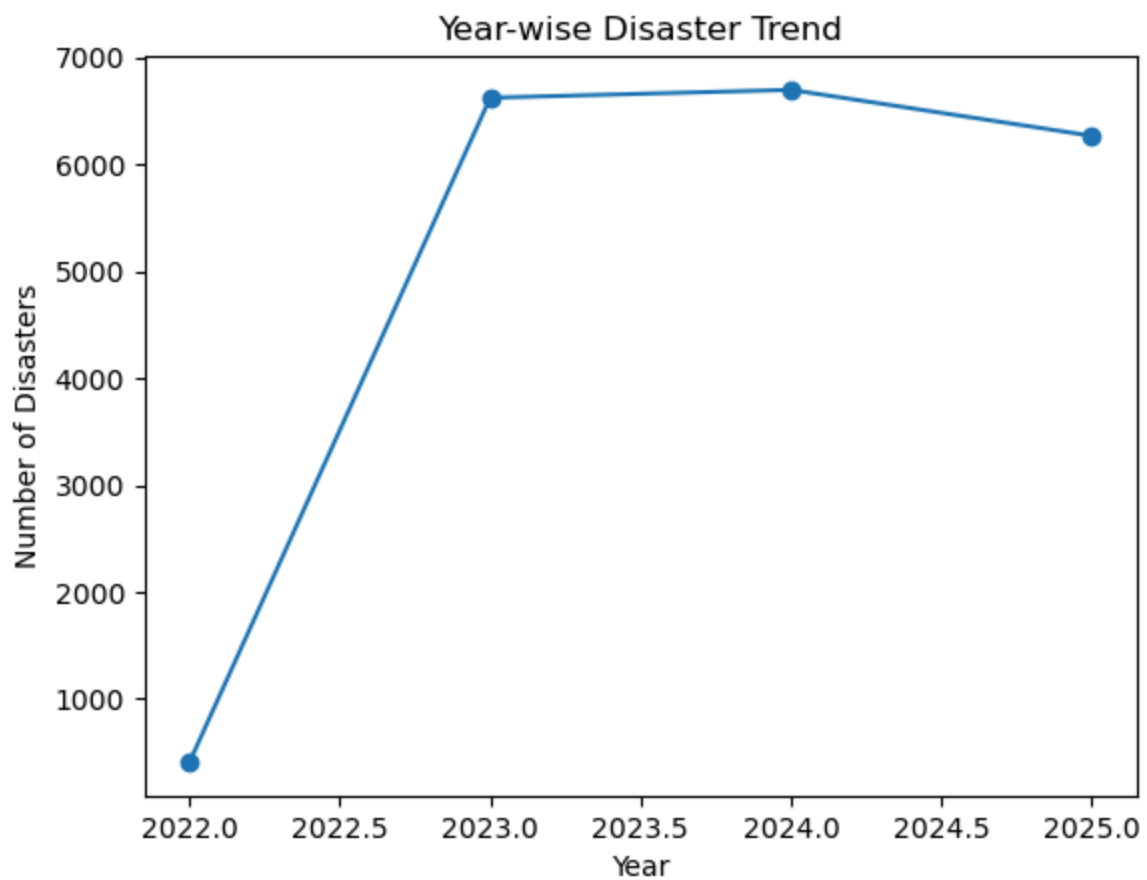


Most disasters in the dataset are non-major events, with major disasters forming a smaller but significant portion.

```

In [20]: plt.figure()
df.groupby('Year').size().plot(kind='line', marker='o')
plt.title("Year-wise Disaster Trend")
plt.xlabel("Year")
plt.ylabel("Number of Disasters")
plt.show()

```



Disaster frequency increased significantly after 2022 and remained consistently high in later years.

```
In [21]: fig = px.box(
          df,
          x="disaster_type",
          y="response_time_hours",
          title="Response Time by Disaster Type"
        )
          fig.show()
```

Emergency response time differs across disaster types but typically ranges within a moderate timeframe.

```
In [22]: fig = px.scatter_geo(
    df,
    lat="latitude",
    lon="longitude",
    color="disaster_type",
    size="severity_level",
    hover_name="location",
    title="Global Disaster Hotspots"
)
fig.show()
```

Certain geographic regions act as disaster hotspots where multiple types of disasters frequently occur.

```
In [23]: fig = px.scatter(  
    df,  
    x="severity_level",  
    y="estimated_economic_loss_usd",  
    size="affected_population",  
    color="is_major_disaster",  
    hover_data=["disaster_type", "location"],  
    title="Severity vs Economic Loss (Bubble = Affected Population)"  
)  
fig.show()
```

Severe disasters cause significantly higher economic losses and affect larger populations.

```
In [24]: aid_major = (  
    df.groupby(["aid_provided", "is_major_disaster"])  
        .size()  
        .reset_index(name="count"))  
fig = px.pie(  
    aid_major,  
    names="aid_provided",  
    values="count",  
    hole=0.5,  
    title="Aid Provided Distribution"  
)  
fig.show()
```

Most disaster events receive humanitarian aid, indicating active disaster management support.

```
In [25]: fig = px.violin(
          df,
          x="is_major_disaster",
          y="severity_level",
          box=True,
          points="all",
          color="is_major_disaster",
          title="Severity Distribution: Major vs Non-Major Disasters"
        )
          fig.show()
```

Major disasters are strongly associated with higher severity levels.

```
In [26]: fig = px.scatter(  
    df,  
    x="infrastructure_damage_index",  
    y="estimated_economic_loss_usd",  
    color="is_major_disaster",  
    title="Infrastructure Damage vs Economic Loss"  
)  
fig.show()
```

Greater infrastructure damage leads to higher economic losses, especially during major disasters.

```
In [27]: heatmap_data = (  
    df.groupby(["Year", "disaster_type"])  
    .size()  
    .reset_index(name="count")  
)  
  
fig = px.density_heatmap(  
    heatmap_data,  
    x="Year",  
    y="disaster_type",  
    z="count",  
    color_continuous_scale="Turbo",  
    title="Disaster Frequency Heatmap (Year vs Type)"  
)  
fig.show()
```

Earthquakes remain the most frequent disaster type across multiple years.

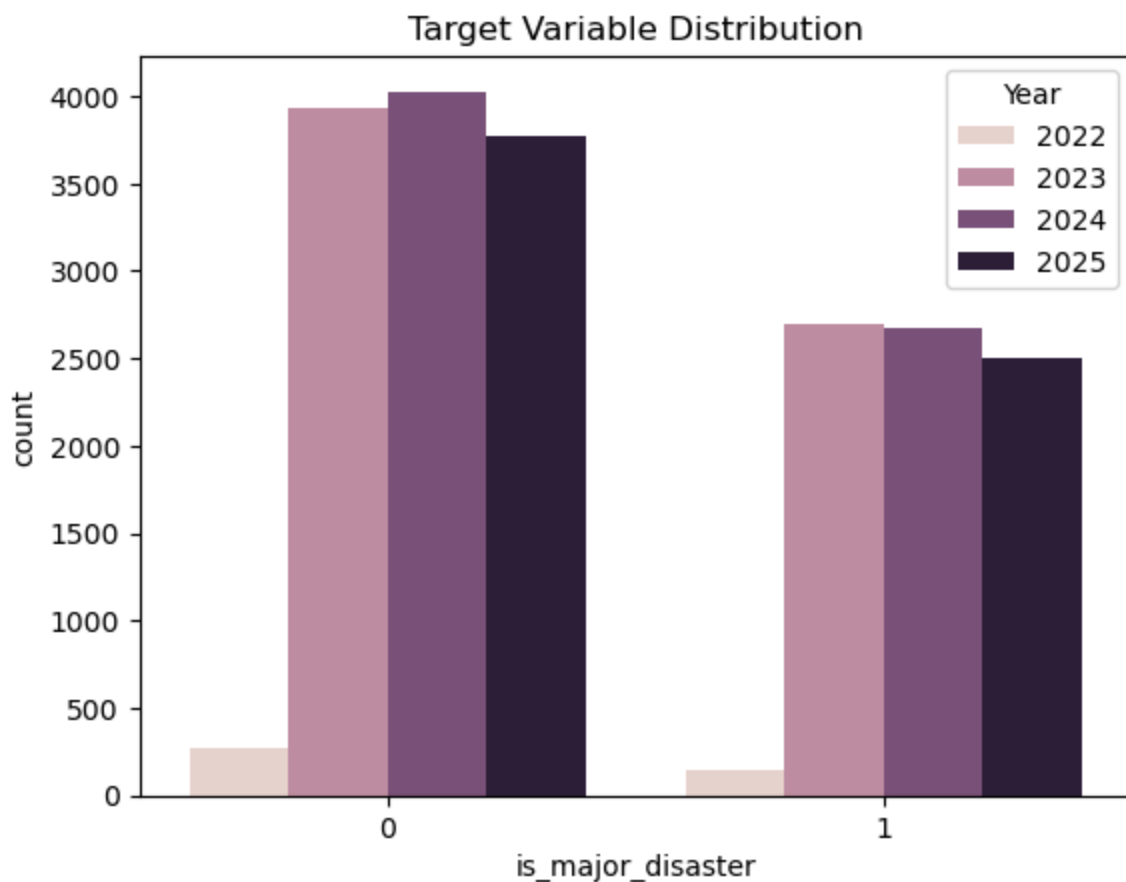
```
In [28]: prob_data = (  
    df.groupby("severity_level")["is_major_disaster"]  
        .mean()  
        .reset_index()  
fig = px.line(  
    prob_data,  
    x="severity_level",  
    y="is_major_disaster",  
    markers=True,  
    title="Probability of Major Disaster by Severity Level")  
fig.show()
```

Higher severity levels strongly increase the probability of a disaster being classified as major.

```
In [29]: fig = px.parallel_coordinates(  
    df,  
    dimensions=[  
        "severity_level",  
        "response_time_hours",  
        "affected_population",  
        "estimated_economic_loss_usd"  
    ],  
    color="is_major_disaster",  
    title="Multivariate Disaster Impact Analysis"  
)  
fig.show()
```

Major disasters show significantly higher impact across multiple variables compared to non-major disasters.

```
In [30]: sns.countplot(x='is_major_disaster', data=df, hue='Year')  
plt.title("Target Variable Distribution")  
plt.show()
```



The target variable distribution is moderately balanced, which helps improve classification model performance.

Label Encoding

Categorical features like disaster type, location, aid provided are converted into numerical format using label encoding.

```
In [31]: label = LabelEncoder()
cat_col = ['disaster_type', 'location', 'aid_provided']
for col in cat_col:
    df[col] = df[col].astype(str)
    df[col] = label.fit_transform(df[col])
```

```
In [32]: df.columns
```

```
Out[32]: Index(['disaster_type', 'location', 'latitude', 'longitude', 'Year',
               'severity_level', 'affected_population', 'estimated_economic_loss_us
               d',
               'response_time_hours', 'aid_provided', 'infrastructure_damage_index',
               'is_major_disaster'],
              dtype='object')
```

Feature Selection (x, y)

```
In [33]: x=df.drop(columns=['is_major_disaster'])  
y=df['is_major_disaster']
```

Train-Test Split

The dataset is divided into 80% training data and 20% testing data.

```
In [34]: x_train,x_test,y_train,y_test=train_test_split(x, y,test_size=0.2,random_state=42)
```

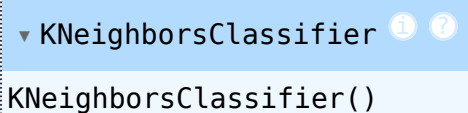
Feature Scaling

KNN is a distance-based algorithm. Feature scaling ensures that variables with larger values do not dominate the distance calculation.

```
In [35]: scaler = StandardScaler()  
x_scaled = scaler.fit_transform(x)
```

Model Training

```
In [36]: knn = KNeighborsClassifier()  
knn.fit(x_train, y_train)
```

```
Out[36]:   
KNeighborsClassifier()
```

Model Prediction

```
In [37]: y_pred = knn.predict(x_test)
```

Model Evaluation

Accuracy Score

```
In [38]: acc = accuracy_score(y_test, y_pred)
```

```
acc
```

```
Out[38]: 0.83725
```

Classification Report

```
In [39]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.85	0.89	0.87	2445
1	0.81	0.76	0.78	1555
accuracy			0.84	4000
macro avg	0.83	0.82	0.83	4000
weighted avg	0.84	0.84	0.84	4000

```
In [40]: print(y_train.unique())  
print(y_test.unique())
```

```
[0 1]
```

```
[0 1]
```

Confusion Matrix

```
In [41]: cm=confusion_matrix(y_test,y_pred)  
print(cm)
```

```
[[2168 277]  
 [ 374 1181]]
```

Cross Validation

Cross-validation evaluates model stability by training and testing the model on multiple data splits.

```
In [42]: from sklearn.model_selection import cross_val_score
```

```
In [43]: score = cross_val_score(knn,x_train,y_train, cv=5)  
print(score)  
print(score.mean()*100)
```

```
[0.8096875 0.8228125 0.8134375 0.8175    0.821875 ]  
81.70625
```

```
In [44]: cv_scores = cross_val_score(knn,x_train,y_train, cv=5, scoring='accuracy')  
print("CV Scores:",cv_scores)
```

```
print("Mean CV Accuracy:", cv_scores.mean())
```

```
CV Scores: [0.8096875 0.8228125 0.8134375 0.8175      0.821875 ]  
Mean CV Accuracy: 0.8170624999999999
```

In []:

GridSearchCV

GridSearchCV tests multiple combinations of hyperparameters to find the best K value and distance metric.

```
In [45]: param_grid = {  
    'n_neighbors': [3, 5, 7, 9, 11],  
    'weights': ['uniform', 'distance'],  
    'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],  
    'p': [1, 2]  
}  
  
knn = KNeighborsClassifier()  
grid_search = GridSearchCV(estimator=knn, param_grid=param_grid, cv=5, scoring=  
grid_search.fit(x_train, y_train)  
print("Best Parameters: ", grid_search.best_params_)  
print("Best CV Accuracy: ", grid_search.best_score_)  
  
best_grid_model = grid_search.best_estimator_
```

```
Best Parameters: {'algorithm': 'auto', 'n_neighbors': 11, 'p': 2, 'weights':  
'uniform'}  
Best CV Accuracy: 0.8246249999999999
```

In []:

RandomizedSearchCV

RandomizedSearchCV is used to optimize the hyperparameters of the K-Nearest Neighbors model by randomly sampling combinations of parameters from a predefined distribution.

```
In [46]: knn = KNeighborsClassifier()  
random_search = RandomizedSearchCV(estimator=knn, param_distributions=param_gr  
    scoring='accuracy', n_jobs=-1)  
  
random_search.fit(x_train, y_train)  
print("Best Parameters: ", random_search.best_params_)  
print("Best CV Accuracy: ", random_search.best_score_)  
best_random_model = random_search.best_estimator_
```

Best Parameters: {'weights': 'uniform', 'p': 2, 'n_neighbors': 11, 'algorithm': 'kd_tree'}
Best CV Accuracy: 0.8246249999999999

```
In [47]: knn = KNeighborsClassifier()
knn .fit(x_train, y_train)
y_prob = knn.predict_proba(x_test)[: , 1]
```

```
In [ ]:
```

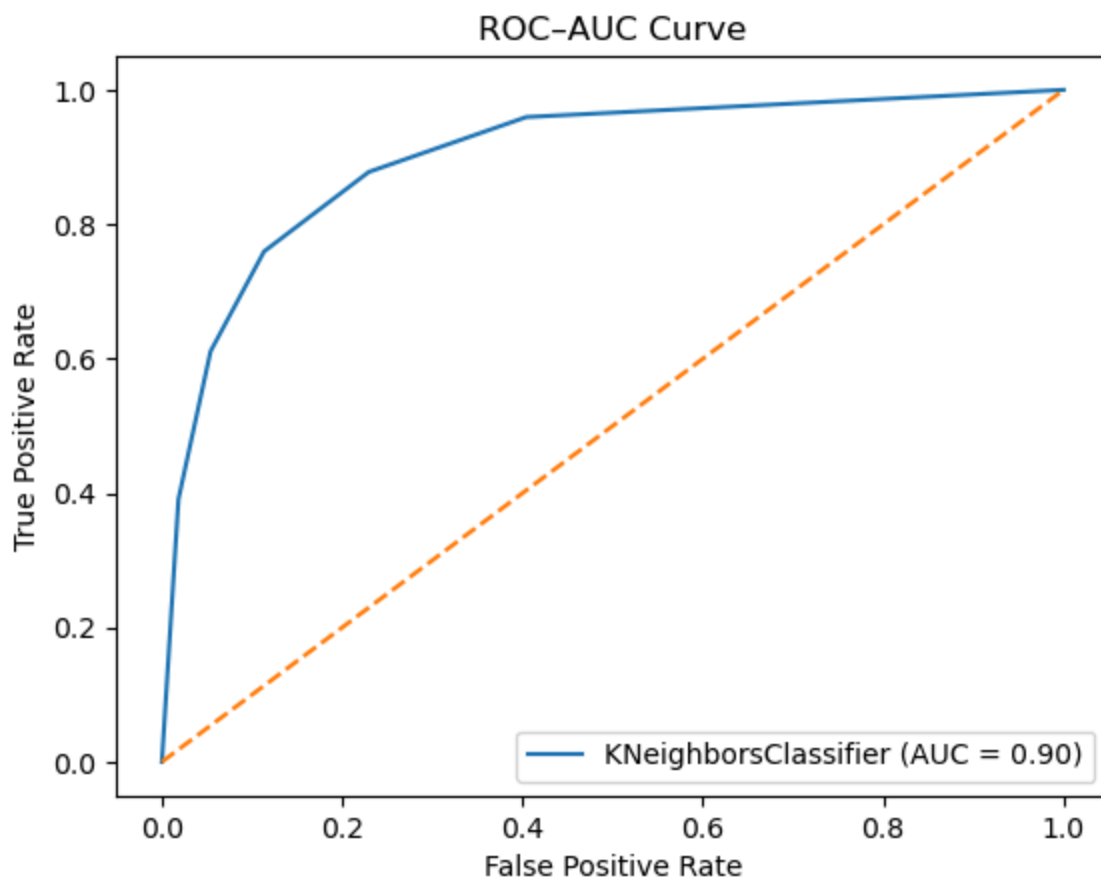
ROC-AUC

The Receiver Operating Characteristic (ROC) curve is used to evaluate the performance of a classification model at different classification thresholds. The Area Under the Curve (AUC) provides a single metric that summarizes the model's ability to distinguish between the classes. A higher AUC score indicates better classification performance.

```
In [48]: from sklearn.metrics import roc_auc_score, roc_curve
```

```
In [49]: fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)
```

```
In [50]: plt.figure()
plt.plot(fpr, tpr, label=f'KNeighborsClassifier (AUC = {auc_score:.2f})')
plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-AUC Curve')
plt.legend()
plt.show()
```



High AUC values indicate strong classification capability.

In []:

In []:

In []:

In []:

Improving Model Performance with Advanced Algorithms

In []:

```
In [58]: # Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
```

```

from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# Evaluation function
def evaluate(y_true, y_pred):
    return (
        accuracy_score(y_true, y_pred),
        precision_score(y_true, y_pred, average='weighted'),
        recall_score(y_true, y_pred, average='weighted'),
        f1_score(y_true, y_pred, average='weighted')
    )

results = []

# Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(x_train, y_train)
y_pred = lr.predict(x_test)
results.append(["Logistic Regression", *evaluate(y_test, y_pred)])

# Decision Tree
dt = DecisionTreeClassifier()
dt.fit(x_train, y_train)
y_pred = dt.predict(x_test)
results.append(["Decision Tree", *evaluate(y_test, y_pred)])

# Random Forest
rf = RandomForestClassifier()
rf.fit(x_train, y_train)
y_pred = rf.predict(x_test)
results.append(["Random Forest", *evaluate(y_test, y_pred)])

# Gradient Boosting
gb = GradientBoostingClassifier()
gb.fit(x_train, y_train)
y_pred = gb.predict(x_test)
results.append(["Gradient Boosting", *evaluate(y_test, y_pred)])

# AdaBoost
ab = AdaBoostClassifier()
ab.fit(x_train, y_train)
y_pred = ab.predict(x_test)
results.append(["AdaBoost", *evaluate(y_test, y_pred)])

# XGBoost
xgb = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')
xgb.fit(x_train, y_train)
y_pred = xgb.predict(x_test)
results.append(["XGBoost", *evaluate(y_test, y_pred)])

# LightGBM
lgb = LGBMClassifier(verbose=-1)
lgb.fit(x_train, y_train)

```

```

y_pred = lgb.predict(x_test)
results.append(["LightGBM", *evaluate(y_test, y_pred)])

# CatBoost
cb = CatBoostClassifier(verbose=0)
cb.fit(x_train, y_train)
y_pred = cb.predict(x_test)
results.append(["CatBoost", *evaluate(y_test, y_pred)])

# TABLE
columns = ["Model", "Accuracy", "Precision", "Recall", "F1 Score"]

df = pd.DataFrame(results, columns=columns)
df = df.sort_values(by="Accuracy", ascending=False)

display(df)

```

	Model	Accuracy	Precision	Recall	F1 Score
3	Gradient Boosting	0.96050	0.960509	0.96050	0.960403
6	LightGBM	0.95900	0.959001	0.95900	0.958899
7	CatBoost	0.95750	0.957477	0.95750	0.957406
2	Random Forest	0.95750	0.957600	0.95750	0.957351
5	XGBoost	0.95700	0.956948	0.95700	0.956938
4	AdaBoost	0.95700	0.957094	0.95700	0.957035
1	Decision Tree	0.93700	0.937381	0.93700	0.937120
0	Logistic Regression	0.92325	0.923094	0.92325	0.923134

In []:

```

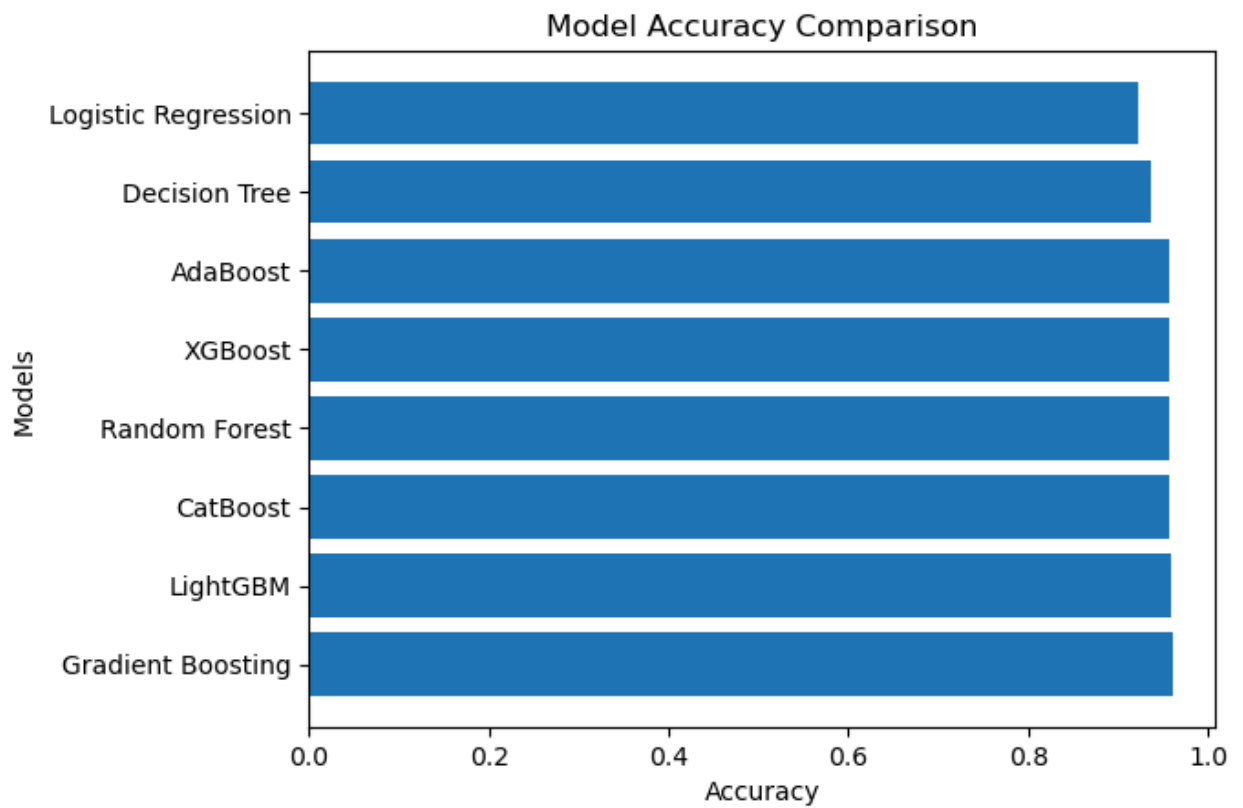
In [59]: plt.figure()

plt.barh(df["Model"], df["Accuracy"])

plt.xlabel("Accuracy")
plt.ylabel("Models")
plt.title("Model Accuracy Comparison")

plt.show()

```



Most models perform very similarly with high accuracy, but ensemble methods (Gradient Boosting, LightGBM, CatBoost, Random Forest, XGBoost) slightly outperform simpler models like Logistic Regression.

In []:

In []:

In []:

In []:

In []:

In []: